

PCMP

Predictive Canonical Monotonic Permutation

Version 1 Specification

Draft — Frozen Semantics

1. Scope & Non-Goals

NORMATIVE

1.1 Scope

NORMATIVE

This specification defines a canonical, deterministic, and reversible binary representation for finite sequences of IEEE-754 binary32 (float32) values, together with an associated permutation and an optional predictive transform.

This specification applies only to:

- IEEE-754 binary32 floating-point values
- Little-endian byte representation
- Sequences whose elements are treated as opaque 32-bit bit patterns

1.2 Explicit Non-Goals

NORMATIVE

This specification does NOT define or guarantee:

- Numerical equivalence
- Floating-point arithmetic determinism
- Preservation of NaN payloads beyond bitwise identity
- Canonicalization of floating-point computations
- Cross-precision behavior (e.g., float64)
- Textual or JSON representations
- Performance characteristics

1.3 Guarantees

NORMATIVE

A conforming implementation MUST guarantee:

- Bit-exact canonical ordering
- Deterministic output for identical input bit patterns
- Reversibility via the defined permutation
- Fail-closed verification semantics

2. Terminology & Normative Language

NORMATIVE

The key words MUST, MUST NOT, SHOULD, MAY, and SHALL are to be interpreted as described in RFC 2119.

Definitions

Canonical

A representation is canonical if and only if all conforming implementations produce bit-identical output for the same admissible input.

Producer

An implementation that generates PCMP-encoded data.

Verifier

An implementation that validates PCMP-encoded data against this specification.

Conforming Implementation

An implementation that satisfies all MUST requirements herein.

Admissible Input

A finite sequence of 32-bit values, each treated as an IEEE-754 binary32 bit pattern, including NaNs and infinities.

3. Data Model

NORMATIVE

- Input is a finite ordered sequence of 32-bit values.
- Each element is interpreted only as a 32-bit bit pattern.
- No implicit normalization, rounding, or NaN canonicalization is permitted.
- Order of elements is significant.

4. Canonical Ordering Definition

CRITICAL, NORMATIVE

4.1 Canonical Mapping

NORMATIVE

Each IEEE-754 binary32 value SHALL be mapped to a 32-bit unsigned integer using the following bijective transformation:

```
Let u be the raw IEEE-754 binary32 bit pattern.

if (u & 0x80000000)
    mapped = ~u;
else
    mapped = u | 0x80000000;
```

This mapping SHALL be applied exactly.

4.2 Ordering Rule

NORMATIVE

Canonical ordering is defined as unsigned integer comparison of the mapped values.

4.3 Special Values

NORMATIVE

This ordering guarantees:

- All negative values precede positive values
- $-\infty < \dots < -0 < +0 < \dots < +\infty$
- All NaN bit patterns are ordered after $+\infty$ according to mapped value
- Signed zero is ordered deterministically
- Subnormals are ordered by magnitude

No alternative total order is permitted.

5. Predictive Transform Semantics

NORMATIVE

Predictive transforms are applied after canonical ordering and permutation application, and SHALL operate exclusively on the mapped uint32 sequence defined in Section 4.

5.1 Predictor Identifiers

NORMATIVE

Order	Meaning
0	No predictor
1	First-order delta
2	Second-order linear predictor

5.2 Arithmetic Rules

NORMATIVE

- All arithmetic is performed on mapped unsigned 32-bit integers
- Overflow behavior SHALL be modulo 2^{32}
- Intermediate values MUST be truncated to 32 bits
- No saturation, clamping, or widening is permitted

5.3 Order-1 Transform

NORMATIVE

```
out[0] = u[0]
out[i] = (u[i] - u[i-1]) mod  $2^{32}$     for  $i \geq 1$ 
```

5.4 Order-2 Transform

NORMATIVE

```
out[0] = u[0]
out[1] = (u[1] - u[0]) mod  $2^{32}$ 
out[i] = (u[i] - (2*u[i-1] - u[i-2])) mod  $2^{32}$     for  $i \geq 2$ 
```

5.5 Inversion

NORMATIVE

Inverse transforms SHALL reverse the above operations exactly and MUST recover the original mapped sequence bit-for-bit.

6. Binary Format Layout (Byte-Exact)

NORMATIVE

6.1 Endianness

NORMATIVE

All multi-byte fields are little-endian.

6.2 Header Layout (PCMP v1)

NORMATIVE

Offset	Size	Field
0	4	Magic = "PCMP"
4	1	Version = 1
5	1	Predictor Order (0, 1, 2)
6	1	Flags (MUST be 0)
7	1	Reserved (MUST be 0)
8	8	Element count (uint64)
16	...	Payload

Any non-zero Flags value MUST cause verification failure.

6.3 Payload Structure

NORMATIVE

Payload SHALL consist of the following components in order:

1. Transformed Data Block

- Length-prefixed
- Zstandard-compressed
- Contains predictive-transformed uint32 values

2. Permutation Block

- Maps canonical-order indices to original input indices
- Length-prefixed
- Zstandard-compressed
- Contains delta-encoded, unsigned LEB128 permutation indices

3. Proof Metadata

- proof_type (uint64)
- total_n (uint64)
- chunk_bytes (uint64)
- num_chunks (uint64)
- ordering_mode (uint32)
- Stored Merkle root (32 bytes)

4. Footer

- footer_magic (uint32)
- footer_version (uint32)

7. Compression Treatment

NORMATIVE

- Compression IS part of the canonical form
- Compression algorithm: Zstandard (Zstd)
- Compression level is implementation-defined but **MUST** be deterministic
- Decompression **MUST** reproduce the exact original byte stream
- Verification **MUST** occur after decompression
- Failure to decompress **MUST** cause verification failure

8. Cryptographic Binding

NORMATIVE

- Hash algorithm: SHA-256
- Hash input: the uncompressed predictive-transformed uint32 data stream
- Hashing occurs prior to compression
- Header fields, permutation data, and compression framing **MUST NOT** be included
- Verification **MUST** fail on hash mismatch or absence
- No claims beyond collision resistance are made

9. Verification Algorithm

NORMATIVE

A verifier **MUST** perform the following steps in order:

- **Step 1:** Validate magic and version
- **Step 2:** Reject unknown versions
- **Step 3:** Validate header fields
- **Step 4:** Decompress payload
- **Step 5:** Validate element count consistency
- **Step 6:** Compute cryptographic hash over decompressed predictive data
- **Step 7:** Validate cryptographic hash
- **Step 8:** Apply inverse predictor
- **Step 9:** Validate canonical ordering
- **Step 10:** Validate permutation reversibility (applying the permutation reconstructs the original input order)

Any failure **SHALL** cause immediate hard failure.

10. Deterministic Computation Boundary Conditions

NORMATIVE

10.1 Zero-Entropy Fixed Point Theorem

NORMATIVE

Let P be a producer that emits a finite ordered sequence S of IEEE-754 binary32 values.

Assume:

- P is deterministic
- P is race-free and side-effect-free
- P introduces no external entropy

Let C be the PCMP canonicalization, permutation, and predictive transform defined in this specification.

Then: $C(S)$ is a fixed point under replay, permutation, and verification.

10.2 Corollary — Impossibility of Adversarial Unscrambling

NORMATIVE

Under the above conditions, no procedure operating solely on PCMP-encoded artifacts can distinguish between repeated executions of P .

10.3 Practical Implication

NORMATIVE

PCMP exists solely to make the producer's commitment irreversible and verifiable.

11. Versioning Rules

NORMATIVE

- Version 1 is frozen forever
- Future versions **MUST** use a new version number
- Forward compatibility is explicitly disallowed
- Unknown versions **MUST** cause hard failure

12. Failure Semantics (Fail-Closed)

NORMATIVE

On any violation:

- A verifier MUST NOT emit partial output
- A verifier MUST NOT attempt recovery
- A verifier MUST signal failure clearly

13. Test Vectors

RECOMMENDED

The specification SHOULD include canonical float sequences and expected hashes.

14. Threat Model

RECOMMENDED

The system assumes hostile inputs and defends against reordering attacks.

15. Security Considerations

RECOMMENDED

PCMP provides integrity and determinism, not confidentiality or authenticity.

Final Litmus Test

A hostile, competent engineer can implement this specification without seeing the reference code and still produce the same hash.